Regular Expressions

Pattern Matching for Text Processing

■ LEARNING OUTCOME

By the end of this module you will write regular expressions to find, extract, validate, and replace text patterns -- using Python's re module for everything from simple searches to complex data extraction.

■ Skills You Will Gain
• Understand regex metacharacters and quantifiers
• Use re.search(), re.match(), re.findall(), re.sub()
• Write character classes and groups
• Use lookaheads and lookbehinds
• Compile patterns for performance
• Apply regex for validation, extraction, and parsing

■ **Regular expressions are a superpower for text processing. Email validation, log parsing, data extraction, URL matching -- regex solves in one line what would take 50 lines of string manipulation code.**

SECTION 1

Regex Syntax Reference

Pattern	Matches
.	Any character except newline
\\d	Digit [0-9]
\\D	Non-digit
\\w	Word character [a-zA-Z0-9_]
\\W	Non-word character
\\s	Whitespace (space, tab, newline)
\\S	Non-whitespace
\\b	Word boundary
^	Start of string (or line with MULTILINE)
$	End of string (or line with MULTILINE)
[abc]	Character class -- a, b, or c
[^abc]	Negated class -- NOT a, b, or c
[a-z]	Range -- lowercase letters
(abc)	Capture group
(?:abc)	Non-capturing group
a|b	Alternation -- a OR b
*****	0 or more (greedy)
+	1 or more (greedy)
?	0 or 1 (optional)
{n}	Exactly n times
{n,m}	Between n and m times
***?**	0 or more (lazy -- minimal match)
(?=abc)	Lookahead -- followed by abc
(?!abc)	Negative lookahead
(?<=abc)	Lookbehind -- preceded by abc

SECTION 2

re Module Functions

```
import re text = "Alice (28) lives in Mumbai. Bob (35) lives in Delhi." # re.search() -- find FIRST match anywhere in string m = re.search(r"\d+", text) if m: print(m.group()) # "28" (first number found) print(m.start()) # 7 (start index) print(m.end()) # 9 (end index) print(m.span()) # (7,9) # re.match() -- match at BEGINNING of string only m = re.match(r"Alice", text) # matches m = re.match(r"Bob", text) # None (not at start) # re.fullmatch() -- match ENTIRE string m = re.fullmatch(r"\d{3}-\d{4}", "123-4567") # True m = re.fullmatch(r"\d{3}-\d{4}", "ABC-1234") # None # re.findall() -- find ALL matches, returns list numbers = re.findall(r"\d+", text) # ["28","35"] names = re.findall(r"[A-Z][a-z]+", text) # ["Alice","Mumbai","Bob","Delhi"] # re.findall() with groups -- returns list of tuples pattern = r"(\w+) \((\d+)\)" matches = re.findall(pattern, text) # [("Alice","28"),("Bob","35")] # re.finditer() -- iterator of Match objects (memory efficient) for m in re.finditer(r"(\w+) \((\d+)\)", text): name, age = m.groups() print(f"{name} is {age} years old") # re.sub() -- find and replace result = re.sub(r"\d+", "##", text) # replace all numbers with ## result = re.sub(r"\d+", "##", text, count=1) # replace only first # re.sub() with function def double_num(m): return str(int(m.group()) * 2) result = re.sub(r"\d+", double_num, text) # "Alice (56) lives..." # re.split() -- split on pattern parts = re.split(r"[.!?]+\s*", text) # split on sentence endings words = re.split(r"\s+", " hello world ") # split on whitespace # COMPILE -- reuse pattern for performance phone_pattern = re.compile(r"\d{10}", re.IGNORECASE) matches = phone_pattern.findall(large_text) valid = phone_pattern.fullmatch("9876543210")
```

SECTION 3

Groups, Flags and Real Examples

```
# NAMED GROUPS -- (?P<name>pattern) date_pattern = re.compile( r"(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})" ) m = date_pattern.search("Event on 2024-01-15") if m: print(m.group("year")) # "2024" print(m.group("month")) # "01" print(m.groupdict()) # {"year":"2024","month":"01","day":"15"} # FLAGS # re.IGNORECASE (re.I) -- case insensitive # re.MULTILINE (re.M) -- ^ and $ match line start/end # re.DOTALL (re.S) -- . matches newline too # re.VERBOSE (re.X) -- allow whitespace and comments email_pattern = re.compile(r""" ^ # start of string [\w.+-]+ # local part: letters, digits, . + - @ # @ symbol [\w-]+ # domain name (?:\.[\w-]+)* # optional subdomains \.[a-zA-Z]{2,} # TLD (at least 2 chars) $ # end of string """, re.VERBOSE) # COMMON VALIDATION PATTERNS patterns = { "email": r"^[\w.+-]+@[\w-]+\.[a-zA-Z]{2,}$", "phone_IN": r"^[6-9]\d{9}$", # Indian mobile "pincode": r"^[1-9]\d{5}$", # Indian pincode "url": r"^https?://[\w.-]+(:\d+)?(/\S*)?$", "date_iso": r"^\d{4}-(0[1-9]|1[0-2])-(0[1-9]|[12]\d|3[01])$", "pan_card": r"^[A-Z]{5}[0-9]{4}[A-Z]$", # Indian PAN "aadhaar": r"^[2-9]{1}\d{11}$", # 12-digit Aadhaar } def validate(value, pattern_name): pattern = patterns.get(pattern_name) return bool(re.match(pattern, value)) if pattern else False # PRACTICAL: Parse log file log_line = "2024-01-15 10:30:45 ERROR user_service Failed to connect: timeout after 30s" log_pattern = re.compile( r"(?P<date>\d{4}-\d{2}-\d{2}) " r"(?P<time>\d{2}:\d{2}:\d{2}) " r"(?P<level>\w+) " r"(?P<service>\w+) " r"(?P<message>.+)" ) m = log_pattern.match(log_line) if m: print(m.groupdict()) # PRACTICAL: Extract data from text html = '<a href="https://example.com">Click</a> <a href="https://test.org">Test</a>' urls = re.findall(r'href="([^"]+)"', html) print(urls) # ["https://example.com","https://test.org"]
```

■ PRACTICE Q & A

Q1. What is the difference between re.search() and re.match()?

> ***A:*** *re.match() only checks at the BEGINNING of the string. re.search() scans the ENTIRE string for a match. Use re.match() when you know the pattern should be at the start; re.search() for anywhere in the string.*

Q2. What is the difference between greedy and lazy quantifiers?

> ***A:*** *Greedy (*, +, {n,m}): match as MUCH as possible. Lazy (*?, +?, {n,m}?): match as LITTLE as possible. With 'texttestmore', '.*' matches the whole thing; '.*?' matches 'text' only.*

Q3. What is a raw string in Python and why use it for regex?

> ***A:*** *r'string' treats backslashes literally -- \n is two characters, not newline. In regex, \d means 'digit'. Without raw string, you'd need to write '\\d' to pass a literal backslash. Always use raw strings r'' for regex patterns.*

Q4. What is a named group in regex?

A: *(?P<name>pattern) assigns a name to a capturing group. Access with m.group('name') or m.groupdict(). Makes complex patterns self-documenting and more maintainable than numbered groups.*

Q5. When should you compile a regex pattern?

A: *Compile with re.compile() when you use the same pattern multiple times. Compilation converts the pattern string to an internal representation -- faster than compiling on every call. Essential in loops or frequently-called functions.*

■ Regex Cheat Sheet	
`re.search(r'\d+', text)`	Find first match anywhere
`re.match(r'^\d+', text)`	Match at START of string
`re.findall(r'\d+', text)`	List of all matches
`re.finditer(r'\d+', text)`	Iterator of Match objects
`re.sub(r'\d+', '##', text)`	Replace all matches
`re.split(r'\s+', text)`	Split on pattern
`re.compile(r'pattern', re.I)`	Compile for reuse
`m.group() / m.groups()`	Get match / all groups
`m.groupdict()`	Named groups as dict
`(?P<name>...)`	Named capture group
`(?:...)`	Non-capturing group
`r'pattern'`	Always use raw strings for regex